

E - Commerce Recommendation pipeline

REVIEW – 3

Batch 19 - Section 4

Team members

Roll No	Name
---------	------

2420030427	Y. Nimisha
2420030498	K. Sanuhya
2420030516	R. Sarvajith varma

Under the guidance of

Dr. Nalla Shirisha

Associate Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Table of contents

- Abstract
- Introduction
- Objectives
- Literature Survey
- System architecture
- Methodology
- Technologies and Tools
- Implementation and Functionality

Abstract

E-commerce platforms provide a very large number of products, which can make it difficult for customers to find products that match their needs and interests. To solve this problem, recommendation systems study different types of user activities, such as searches, clicks, product views, ratings, cart activity, and previous purchases. These activities help the system understand what a customer is interested in. The collected information is then processed through a recommendation pipeline, where the data is collected, cleaned, prepared, and analyzed. Based on this processed data, suitable products are identified, ranked according to their relevance, and recommended to the customer. The recommendations are personalized because they are based on the individual customer's interests and behavior. This project focuses on understanding how an e-commerce recommendation system works, the different stages involved in its data pipeline, and how these personalized recommendations can improve product discovery, customer experience, and make online shopping easier and more convenient.

Introduction

- E-commerce has become a major part of everyday shopping, with platforms offering thousands or even millions of products. With so many choices available, customers may find it difficult to identify products that are relevant to their needs.
- An e-commerce recommendation system helps solve this problem by understanding customer interests and suggesting products that they are likely to prefer. It uses information such as search history, product views, clicks, ratings, cart activity, and previous purchases.
- Behind these recommendations is a recommendation pipeline. It takes user and product data, processes the information, finds relevant products, ranks them, and finally displays the most suitable recommendations to the customer.
- Therefore, recommendation systems help make online shopping more personalized, convenient, and efficient, while also helping e-commerce platforms improve customer engagement and product discovery.

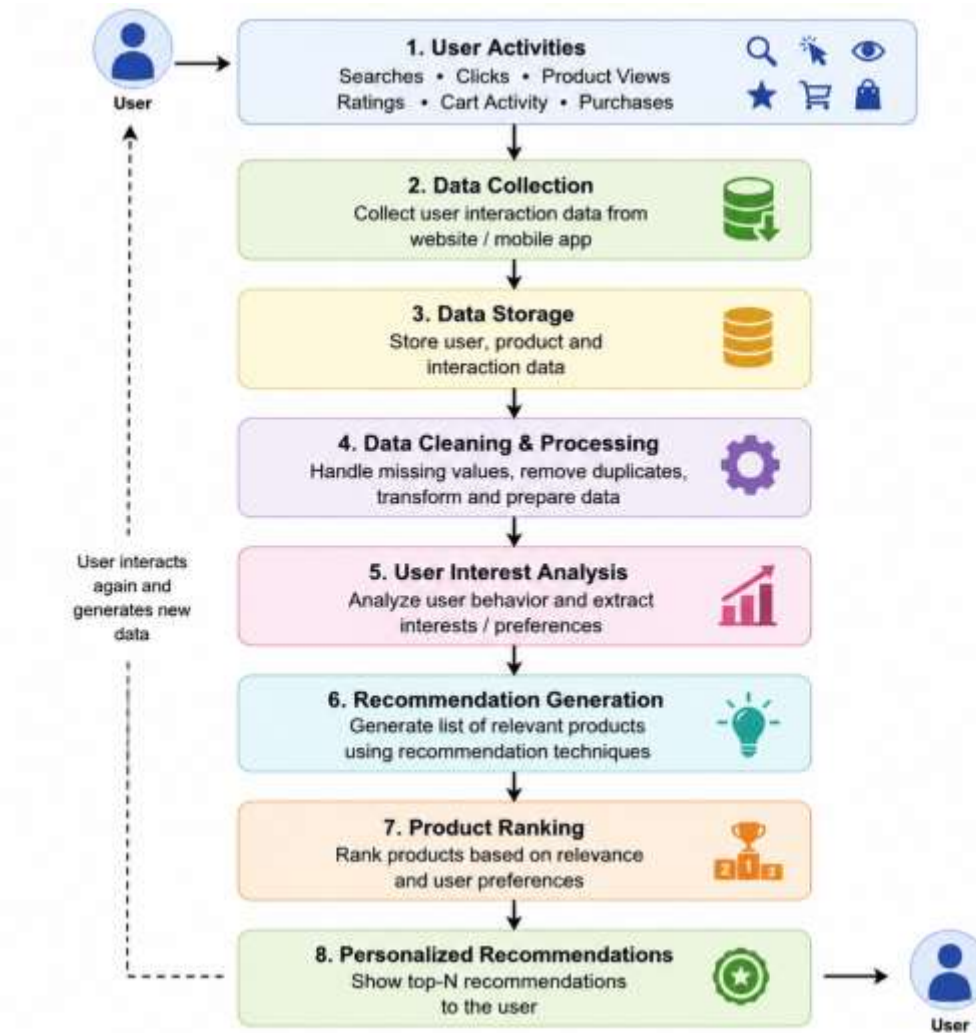
Objectives

- Understand the working of e-commerce recommendation systems
- Collect user and product activity data
- Process and clean recommendation data
- Analyze user interests and preferences
- Generate relevant product recommendations
- Rank products based on user behavior
- Provide personalized recommendations
- Improve product discovery and customer experience

Literature Survey

S.No	Research Paper	Author(s)	Actual Link	Drawback
1.	Toward the Next Generation of Recommender Systems: A Survey	Gediminas Adomavicius, Alexander Tuzhilin	https://doi.org/10.1109/TKDE.2005.99	Provides a broad overview of recommendation techniques, but does not focus on one specific e-commerce application. (Wikipedia)
2.	Amazon.com Recommendations: Item-to-Item Collaborative Filtering	Greg Linden, Brent Smith, Jeremy York	https://doi.org/10.1109/MIC.2003.1167344	Works well for large-scale recommendations, but mainly focuses on item-to-item relationships and may not fully capture complex user preferences. (IEEE Xplore)
3.	Collaborative Filtering for Implicit Feedback Datasets	Yifan Hu, Yehuda Koren, Chris Volinsky	https://doi.org/10.1109/ICDM.2008.22	Uses implicit user feedback such as views and purchases, but implicit feedback does not always indicate whether a user actually liked a product. (CiNii Research)
4.	Matrix Factorization Techniques for Recommender Systems	Yehuda Koren, Robert Bell, Chris Volinsky	https://doi.org/10.1109/MC.2009.263	Provides effective recommendations, but matrix factorization can have difficulty with new users or new products due to the cold-start problem. (OA Monitor Ireland)
5.	Deep Neural Networks for YouTube Recommendations	Paul Covington, Jay Adams, Emre Sargin	https://doi.org/10.1145/2959100.2959190	Uses deep learning for candidate generation and ranking, but requires large amounts of data and computational resources. (DOI)
6.	Neural Collaborative Filtering	Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, Tat-Seng Chua	https://doi.org/10.1145/3038912.3052569	Improves recommendation using neural networks, but the model is more complex and requires more computation than traditional methods. (ACM Digital Library)

System Architecture



Methodology

Collect user and product activity data:

Gather information about user interactions, product details, views, clicks, and purchase history.

Clean and preprocess the data:

Remove missing values, duplicates, and unnecessary data. Convert the data into a suitable format for analysis.

Analyze customer interests and preferences:

Study user behavior to understand their interests and identify products they may prefer.

Generate and rank relevant product recommendations:

Use recommendation algorithms to find suitable products and rank them based on user preferences and product relevance.

Display personalized recommendations:

Present the recommended products to users through the e-commerce platform to improve product discovery and shopping experience.

Technologies and Tools

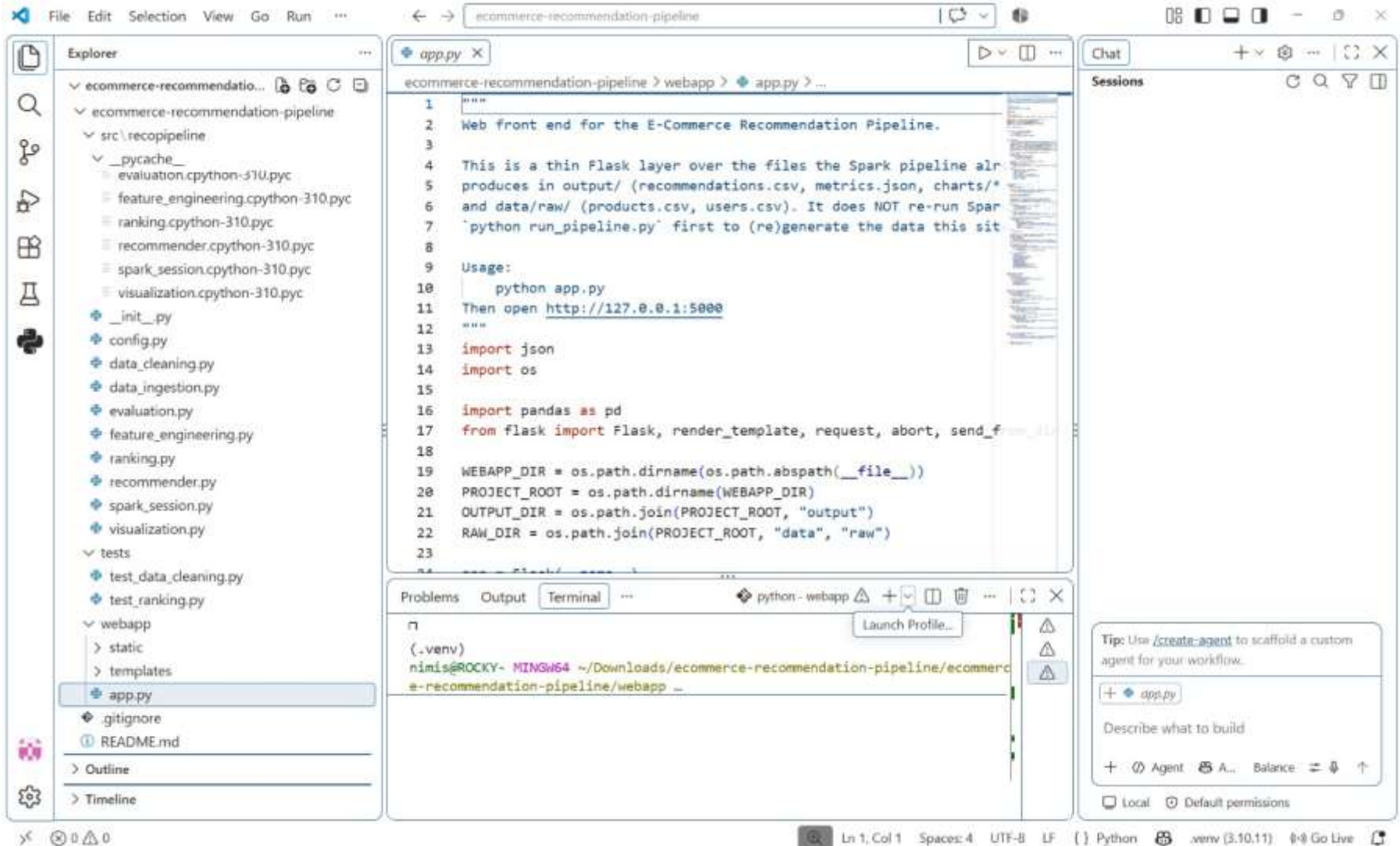
Technologies Used

- Python: Data processing and recommendation logic.
- Pandas: Data cleaning and manipulation.
- NumPy: Numerical operations and data analysis.
- Scikit-learn: Machine learning and recommendation techniques.
- Jupyter Notebook: Development, experimentation, and testing.
- Data Visualization Tools: Understanding user behavior and product trends.

Benefits

- Efficient data processing.
- Better analysis of customer preferences.
- Supports personalized recommendations.
- Helps improve product discovery and user experience.

Implementation



The screenshot displays the Visual Studio Code (VS Code) editor interface. The Explorer panel on the left shows the project structure for 'ecommerce-recommendation-pipeline', including a 'src\recopipeline' directory with various Python files like 'evaluation.cpython-310.pyc', 'feature_engineering.cpython-310.pyc', 'ranking.cpython-310.pyc', 'recommender.cpython-310.pyc', 'spark_session.cpython-310.pyc', and 'visualization.cpython-310.pyc'. The main editor window shows the 'app.py' file, which is a Flask web application. The code includes a docstring describing the app as a thin Flask layer over the Spark pipeline, a 'Usage' section with instructions to run 'python app.py' and open 'http://127.0.0.1:5000', and imports for 'json', 'os', and 'pandas'. The code also defines 'WEBAPP_DIR', 'PROJECT_ROOT', 'OUTPUT_DIR', and 'RAW_DIR'. The Terminal panel at the bottom shows the command prompt in a virtual environment (venv) with the command 'python -m webapp' and the output '(.venv) nimis@ROCKY- MINGW64 ~/Downloads/ecommerce-recommendation-pipeline/ecommerce-recommendation-pipeline/webapp ...'. The Chat panel on the right shows a session with a tip to use '/create-agent' to scaffold a custom agent for the workflow.

```
1 """
2 Web front end for the E-Commerce Recommendation Pipeline.
3
4 This is a thin Flask layer over the files the Spark pipeline alr
5 produces in output/ (recommendations.csv, metrics.json, charts/*
6 and data/raw/ (products.csv, users.csv). It does NOT re-run Spar
7 'python run_pipeline.py' first to (re)generate the data this sit
8
9 Usage:
10     python app.py
11 Then open http://127.0.0.1:5000
12 """
13 import json
14 import os
15
16 import pandas as pd
17 from flask import Flask, render_template, request, abort, send_f
18
19 WEBAPP_DIR = os.path.dirname(os.path.abspath(__file__))
20 PROJECT_ROOT = os.path.dirname(WEBAPP_DIR)
21 OUTPUT_DIR = os.path.join(PROJECT_ROOT, "output")
22 RAW_DIR = os.path.join(PROJECT_ROOT, "data", "raw")
23
```

Tip: Use `/create-agent` to scaffold a custom agent for your workflow.

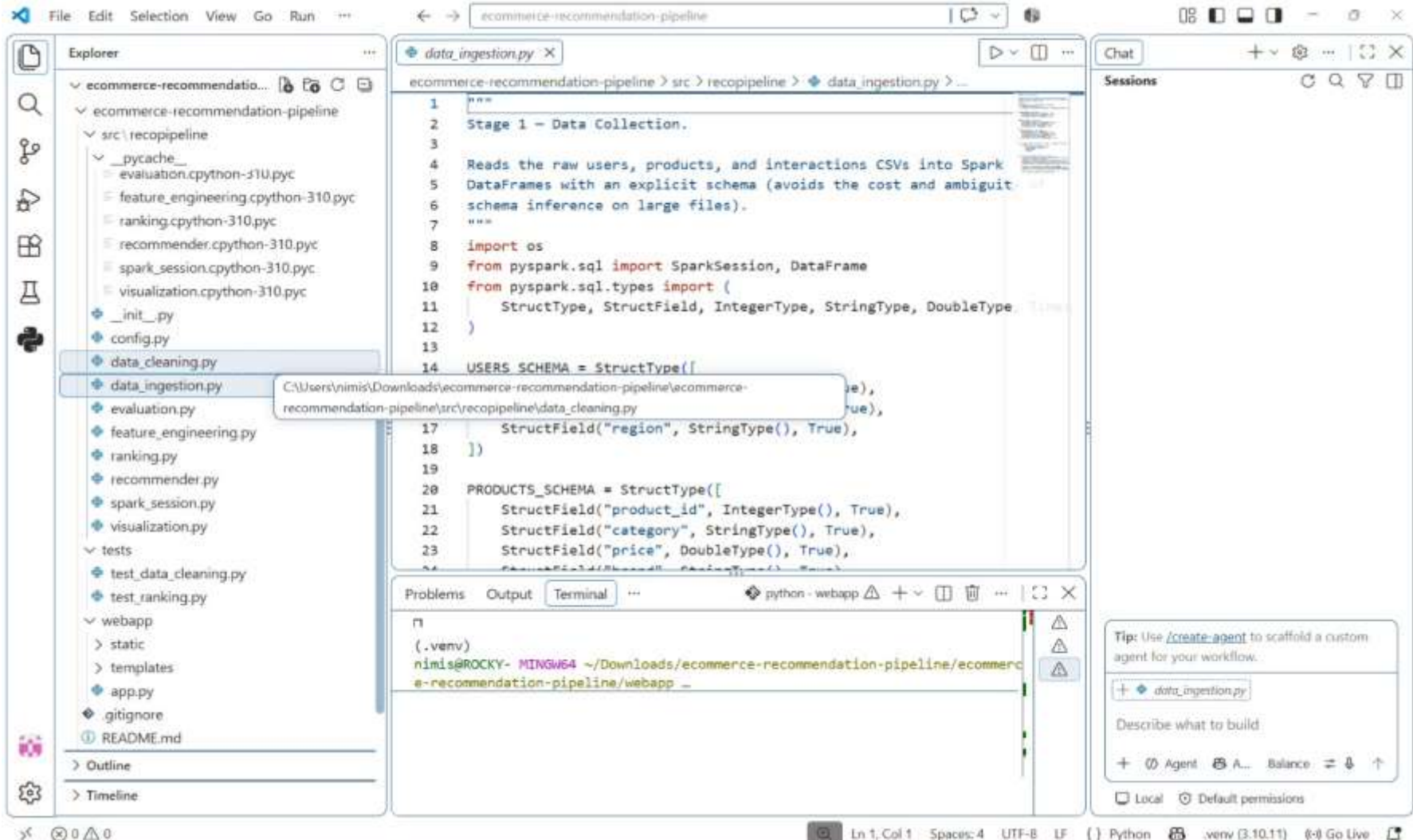
+ `app.py`

Describe what to build

+ Agent A... Balance: $\pm$ $\updownarrow$

☐ Local ☒ Default permissions

Implementation



The screenshot displays the Visual Studio Code (VS Code) interface with the following components:

- Explorer:** Shows the project structure for 'ecommerce-recommendation-pipeline'. The 'src/recpipeline' directory is expanded, listing files like 'data_ingestion.py' (selected), 'data_cleaning.py', 'evaluation.py', 'feature_engineering.py', 'ranking.py', 'recommender.py', 'spark_session.py', and 'visualization.py'. There are also 'tests' and 'webapp' directories.
- Editor:** Displays the code for 'data_ingestion.py'. The code includes a docstring for 'Stage 1 - Data Collection', imports for 'os', 'pyspark.sql', and 'pyspark.sql.types', and defines two schemas: 'USERS_SCHEMA' and 'PRODUCTS_SCHEMA'.
- Terminal:** Shows the command prompt output for the 'python - webapp' command, indicating the current directory is 'C:\Users\nimis\Downloads\ecommerce-recommendation-pipeline\ecommerce-recommendation-pipeline\src\recpipeline\webapp'.
- Chat:** Displays a chat window with a tip: 'Tip: Use /create-agent to scaffold a custom agent for your workflow.' and a list of agents including 'data_ingestion.py'.

```
1 """
2 Stage 1 - Data Collection.
3
4 Reads the raw users, products, and interactions CSVs into Spark
5 DataFrames with an explicit schema (avoids the cost and ambiguity
6 schema inference on large files).
7 """
8 import os
9 from pyspark.sql import SparkSession, DataFrame
10 from pyspark.sql.types import (
11     StructType, StructField, IntegerType, StringType, DoubleType
12 )
13
14 USERS_SCHEMA = StructType([
15     StructField("region", StringType(), True),
16     StructField("product_id", IntegerType(), True),
17     StructField("category", StringType(), True),
18     StructField("price", DoubleType(), True),
19 ])
20
21 PRODUCTS_SCHEMA = StructType([
22     StructField("product_id", IntegerType(), True),
23     StructField("category", StringType(), True),
24     StructField("price", DoubleType(), True),
25 ])
```

Functionality

- **User Activity Collection** – The system collects user searches, clicks, product views, ratings, cart activity, and previous purchases.
- **Data Processing** – The collected user and product data is cleaned and prepared through a recommendation pipeline.
- **User Preference Analysis** – The system analyzes customer behavior to understand their interests and preferences.
- **Product Recommendation** – Based on the analyzed behavior, relevant products are identified and suggested to the user.
- **Product Ranking** – Recommended products are ranked according to their relevance to the user's behavior.
- **Personalized Recommendations** – Different users receive different product suggestions based on their individual activity.
- **Improved Product Discovery** – Users can find relevant products more easily instead of manually searching through thousands or millions of products.
- **Better Shopping Experience** – The overall goal is to make online shopping more personalized, convenient, and efficient.